Experiment 1

1.1 Objective:

Familiarization of Network Environment, Understanding and using network utilities: ipconfig, netstat, ping, telnet, ftp, traceroute etc.

1. **Ipconfig:**

   ipconfig (Windows) or ipconfig (Linux/Unix) is a command-line utility that displays the current TCP/IP network configuration values and refreshes Dynamic Host Configuration Protocol (DHCP) and Domain Name System (DNS) settings.

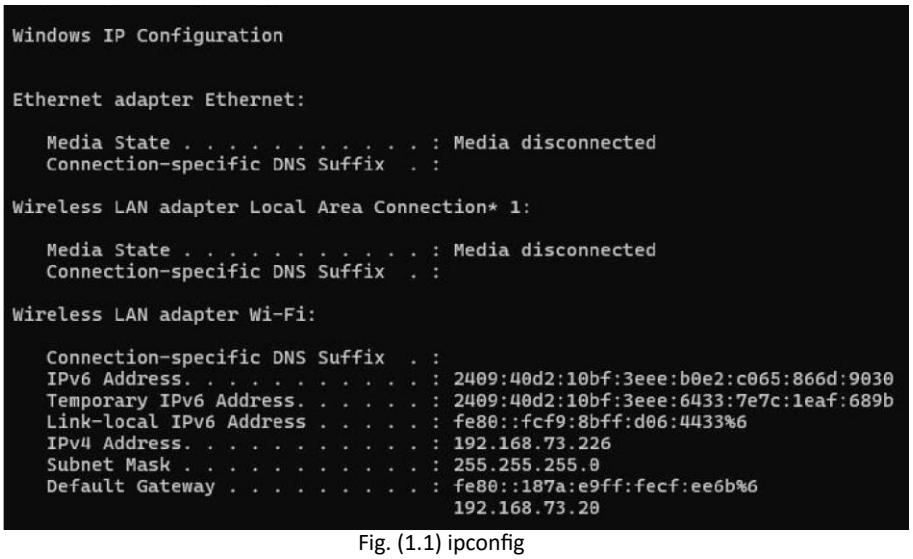


Fig. (1.1) ipconfig

2. **netstat:**

   netstat (network statistics) is a command-line utility that displays network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.

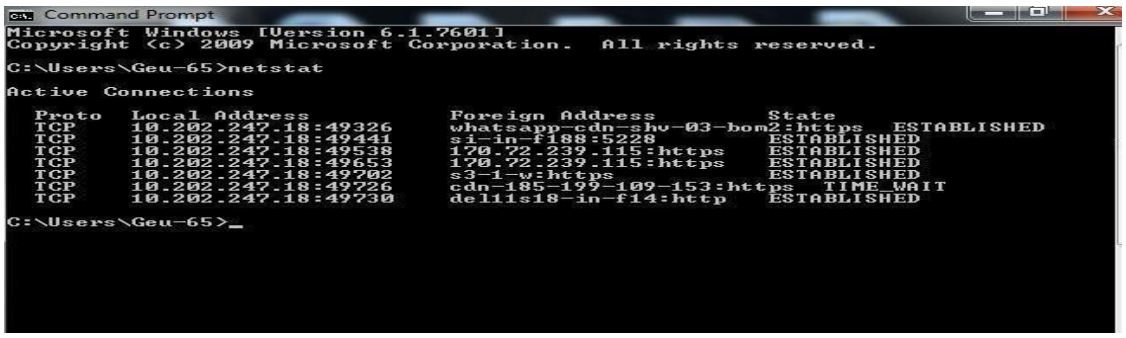


Fig. (1.2) netstat

3. **ping:**

   ping is a computer networks administration software utility used to test the reachability of a host on an Internet Protocol (IP) network.

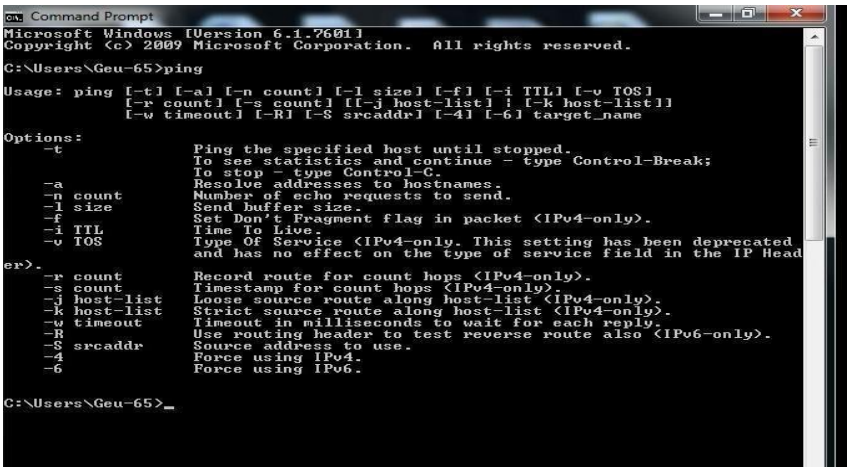


Fig. (1.3) ping

4. **telnet**

Telnet is a remote access CLI tool. If you log into a certain device, you will be presented with the CLI of that device (E.g. switch, router, server, etc.) It allows you to test connectivity to a remote host on a given port, which can help diagnose the issue.

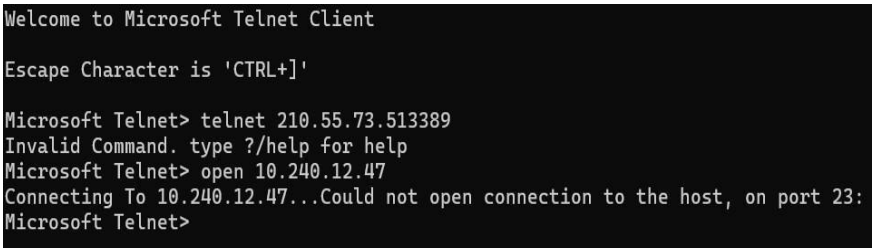


Fig. (1.4) telnet

5. **Traceroute**

Traceroute maps the path that data takes from your computer to a destination on the network, like a website. It shows each step along the way, helping you figure out where delays or failures occur in the network.

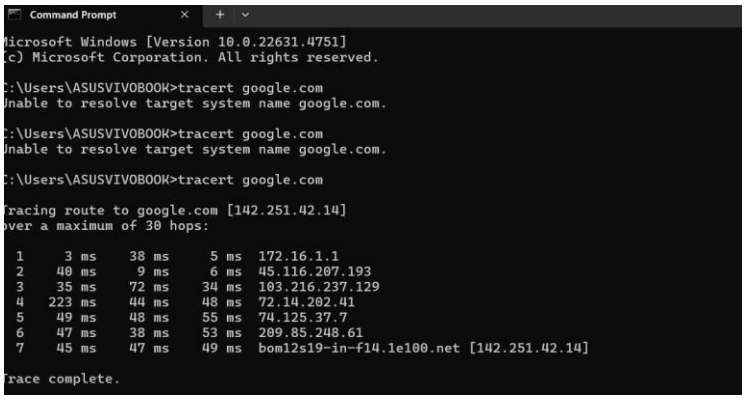


Fig. (1.5) traceroute

1.2 Objective:

Installation and introduction of simulation tool. (Packet Tracer)

Cisco Packet Tracer

Overview:

Cisco Packet Tracer is a powerful network simulation program developed by Cisco Systems. It's designed to help students and professionals practice networking skills, including configuration, troubleshooting, and design.

Key Features:

- **Device Simulation**: Simulate a wide range of network devices including routers, switches, PCs, and IoT devices.
- **Multi-User Collaboration**: Work on the same network topology with others in real-time.
- **Activity Wizard**: Create custom guided learning activities.
- **Supports Protocols**: Work with various networking protocols like OSPF, EIGRP, VLANs, etc.

Installation Steps:

Note: Packet Tracer is available for Windows, macOS, and Linux.

1. **Register for a Cisco Networking Academy Account:**

- Go to the Cisco Networking Academy website.
- Click on "Sign Up" and create a free account.
- You may need to enroll in a free self-paced course (e.g., "Introduction to Packet Tracer") to access the download.

2. **Download Packet Tracer:**

- After logging in, navigate to the Resources or Download Packet Tracer section.
- Select the appropriate version for your operating system.
- Download the installer file.

Fig. (1.6) CISCO Packet Tracer Interface

Experiment 2

Objective:

To configure a basic network topology consisting of routers, switches, and end devices such as PCs or laptops. Configure IP addresses and establish connectivity between devices. (Using packet Tracer)

2.1. **Setting Up Direct Connections Between End Devices Step 1: Set Up Network Devices**

- In Cisco Packet Tracer, place the PCs according to the topology diagram provided (see fig 2.1).
- Use crossover cables to connect the PCs as required.

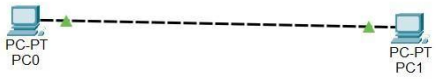


Fig. (2.1) Network Topology

Step 2: Configure IP Addresses

- Select each PC and go to Desktop > IP Configuration.
- Assign the following IP addresses to the PCs:
  - PC0: 192.168.20.10 (see fig 2.2)
  - PC1: 192.168.20.11

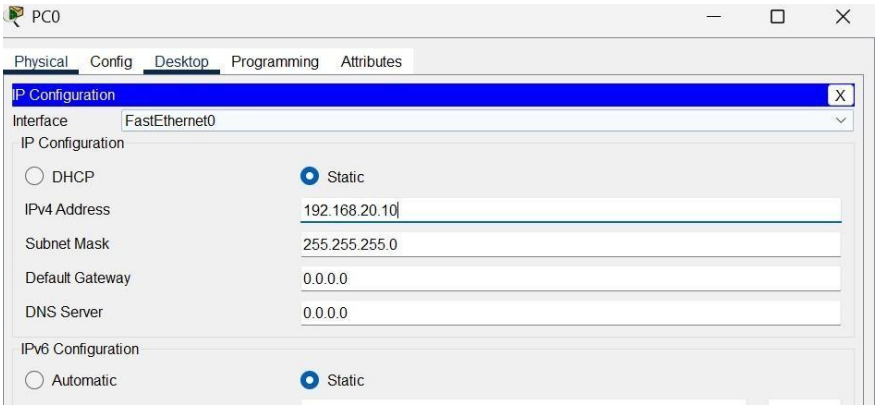


Fig. (2.2) End devices IP config

Step 3: Test Connectivity

- Open the Command Prompt on each PC.
- Use the ping command to check connectivity between the PCs: On PC0, type: ping 192.168.20.11
- A successful ping result should appear like the one shown in fig 2.3

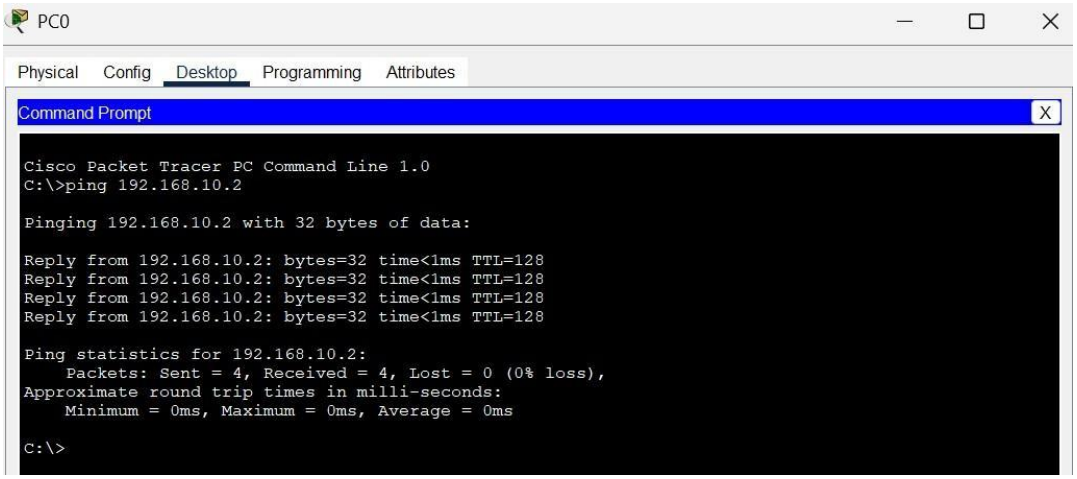


Fig. (2.3) Testing using ping command

2.2. **Direct Connection Between End Devices Using a Switch Step 1: Set Up the Devices**

- Place the PCs and switch in Packet Tracer as shown in fig 2.4.
- Connect each PC to the switch using straight-through Ethernet cables.

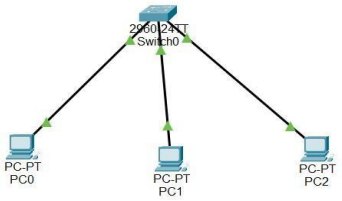


Fig. (2.4) network topology with switch

Step 2: Configure IP Addresses

- Select each PC and go to Desktop > IP Configuration.
- Assign the following IP addresses to the PCs:
  - PC0: 192.168.2.1 (see fig 2.5)
  - PC1: 192.168.2.2 o PC3: 192.168.2.3

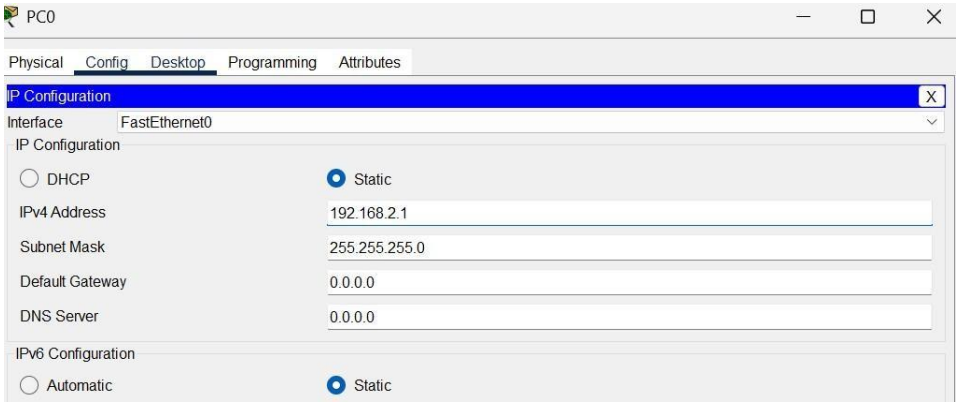


Fig. (2.5) End devices IP config

Step 3: Simulation

- Place the simple PDU on one of the PC and start the simulation.

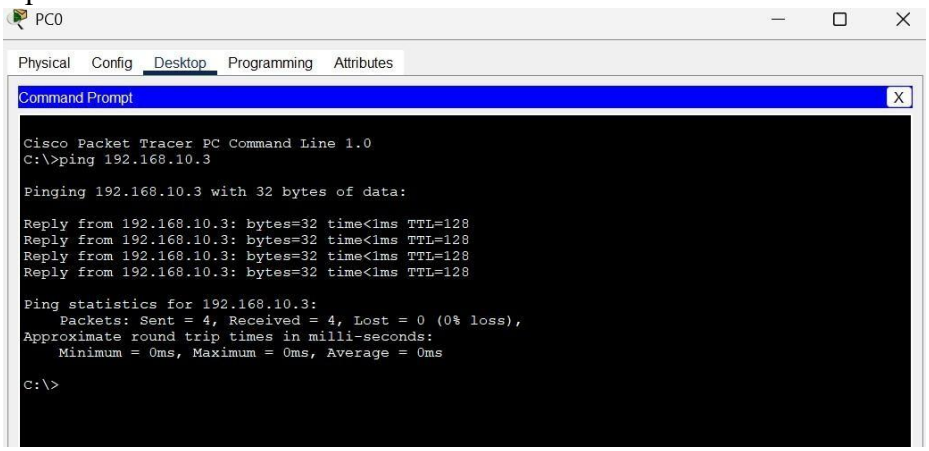


Fig. (2.6) Testing using ping command

2.3. **Connection Between End Devices Using a Hub**

Step 1: Set Up the Devices

- Place the Laptops and Hub in Packet Tracer.
- Connect each Laptop to the Hub using straight-through Ethernet cables.

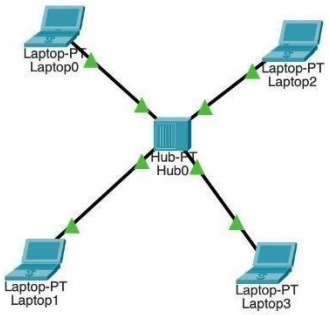


Fig (2.7) Connection between hub and end devices

Step 2: Configure IP Addresses

- Select each Laptop and go to Desktop -> IP Configuration.
- Assign the following IP addresses to the Laptops:
- Laptop0: 192.168.2.4 • Laptop1: 192.168.2.1 • Laptop3: 192.168.2.2
- Laptop2: 192.168.2.3

Step 3: Test Connectivity

- Send a packet from Laptop1 to Laptop2 and track the packet's route.

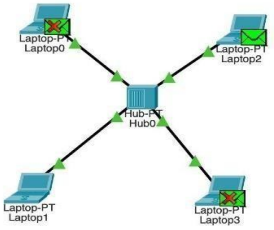


Fig. (2.8) Simulation

Experiment 3

Objective:

To configure a basic network topology consisting of routers, switches, and end devices such as PCs or laptops. Configure IP addresses and establish connectivity between devices. (Using packet Tracer)

Direct Connection Between End Devices Using a Switch and Router Step 1: Set Up the Devices

- Place the PCs, switch and router in packet tracer as shown in fig 1
- Connect each PC to the switch and switch to router using automatically connection cable

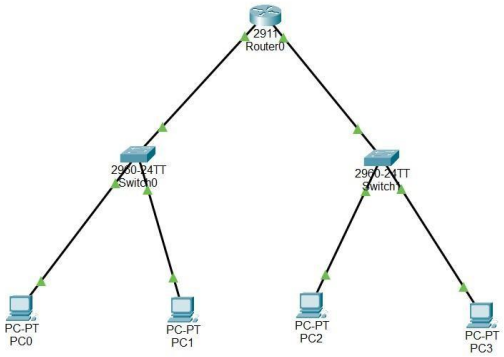


Fig. (3.1) network topology with Router

Step 2: Configure IP Addresses

- Select each PC and go to Desktop > IP configuration.
- Assign the following IP address to the PCs.
- PC0: 192.168.2.1 (see fig 2)
- PC1: 192.168.2.2
- PC3: 10.0.0.1
- PC4: 10.0.0.2
- Now provide the gateways to the PCs accordingly to the router connection.

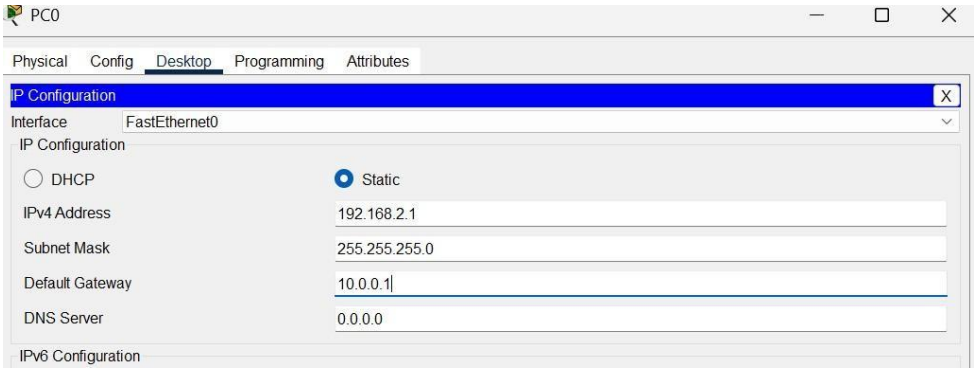


Fig. (3.2) Setting up static IPv4

Step 3: Configure Router & IP Addresses

- Go to Router's CLI and assign IP addresses to each Interface.

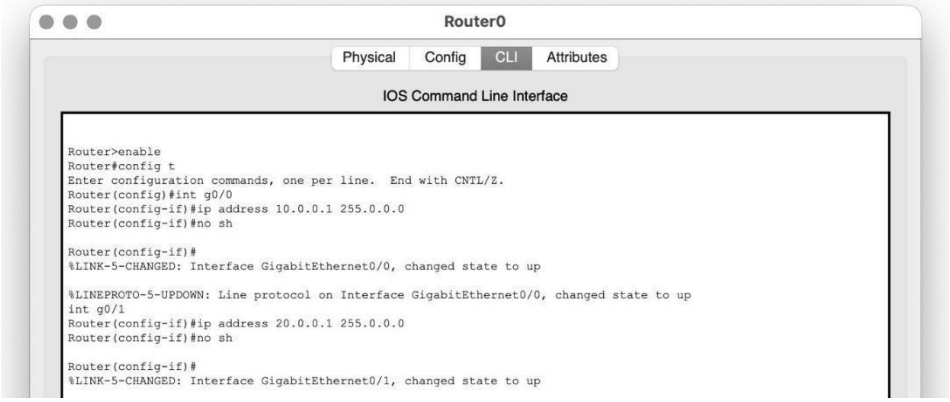


Fig. (3.3) CLI Commands

- Assign IP addresses to each end device and set the default gateway on each to match the router's IP address.

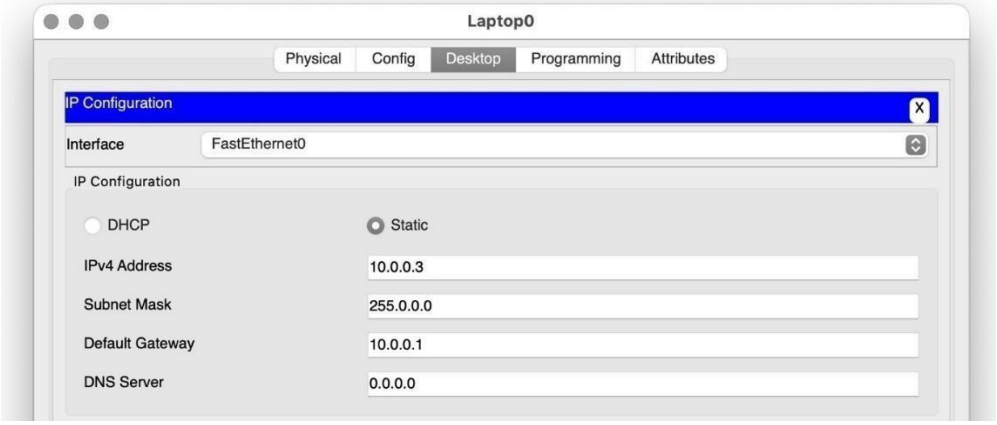


Fig. (3.4) IP Configuration of Desktop

Step 4: Simulation

- Place the simple PDU on first PC of first network to the first PC of second network and start the simulation.

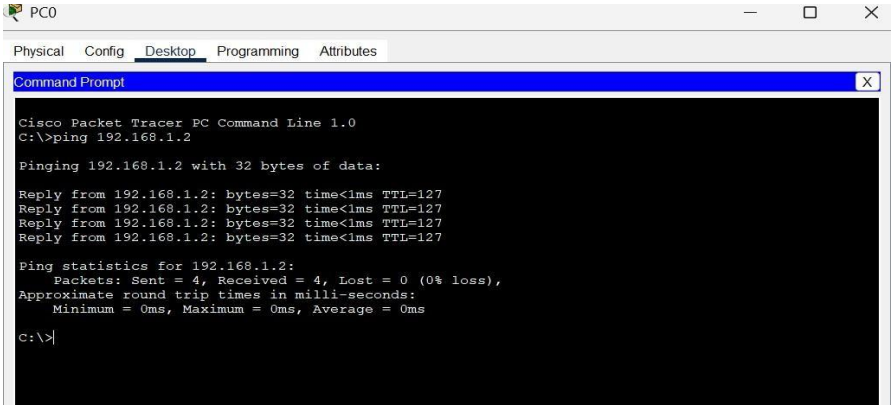


Fig. (3.5) Testing Connectivity using ping command

Experiment 4

(A). Objective: To configure a DHCP server on a router or a dedicated DHCP server device. Assign IP addresses dynamically to devices on the network and verify successful address assignment **Procedure**:

Step 1: Device Setup and Connections

1. Open Packet Tracer.
2. Add Devices: Place the necessary devices (routers, switches, server, and end devices) into the workspace according to the layout in the provided diagram (refer to Image). 3. Connect Devices

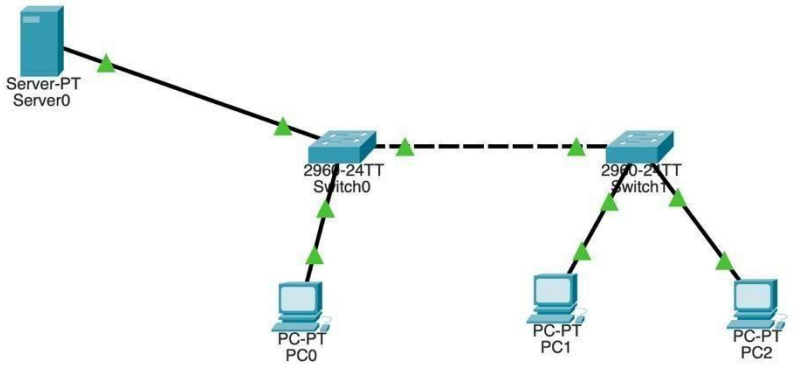
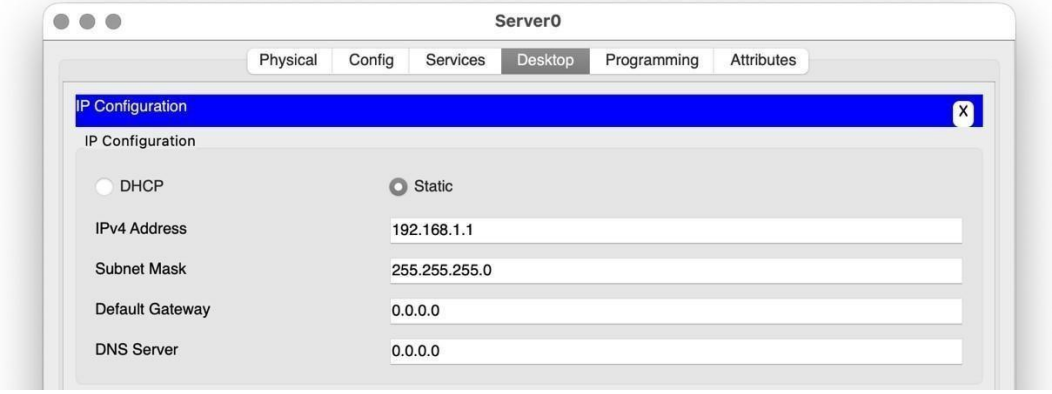


Fig. (4.1) Network Topology

Step 2: Configure IP Address on the Server

- Click on the Server and go to the Desktop tab.
- Configure the IP address and subnet mask for the Server.



- Go to Services -> DHCP, set up the pool and check the Service ‘On’.

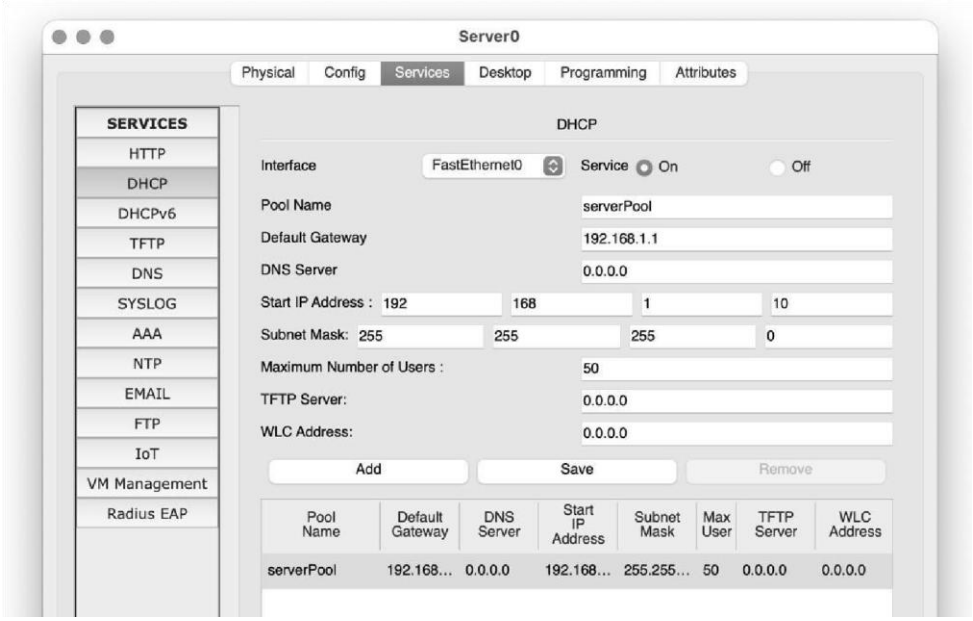


fig (4.2) ServerPool of DHCP in Server **Step 3: Configure the End Devices to Use DHCP.**

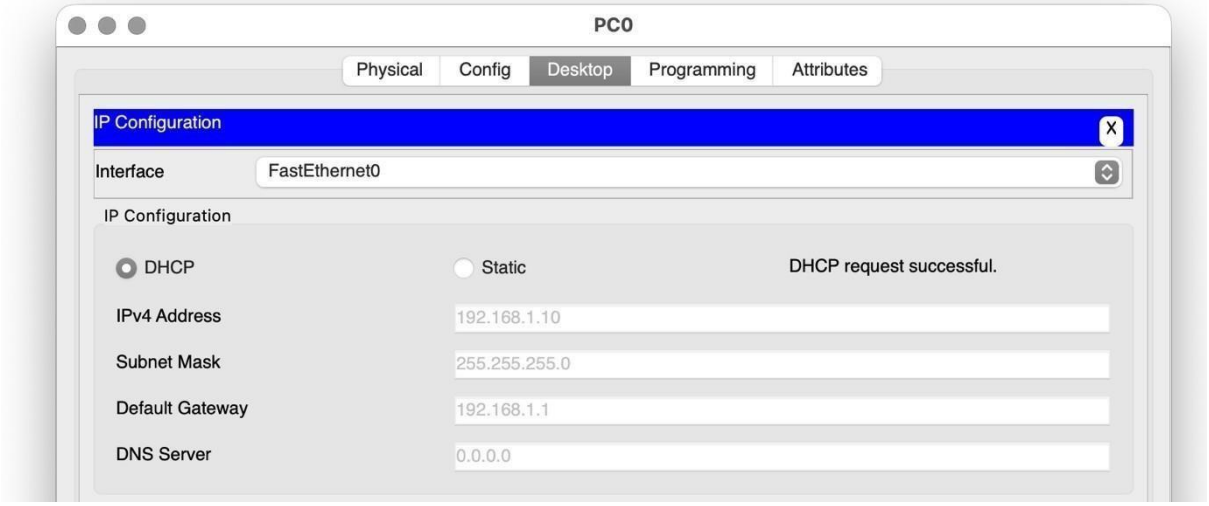


fig (4.3) Ip config of PC

(B). Objective: To configure a DHCP server on dedicated DHCP server device. Assign IP addresses dynamically to devices on the network and verify successful address assignment. (Using packet Tracer)

Step 1: Device Setup and Connections

1. Open Packet Tracer.
2. Add the required devices (server, router, switches and end devices) to the workspace according to the provided diagram.
3. Use straight-through Ethernet cables to connect the devices.

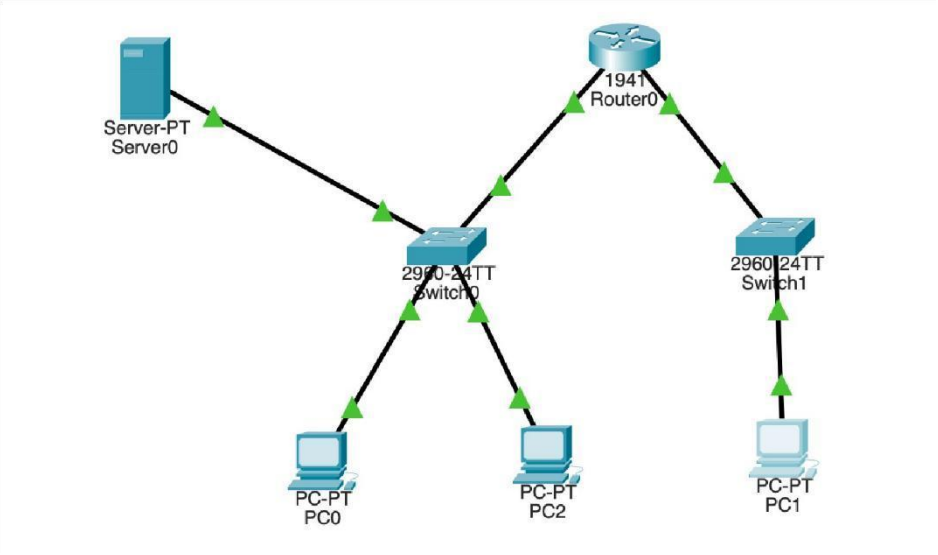


fig (4.4) Configuration of DCHP Server **Step 2: Configure Router & IP Address on the Server.**

- Click on the Server and go to the Desktop tab.
- Configure the IP address and subnet mask for the Server.

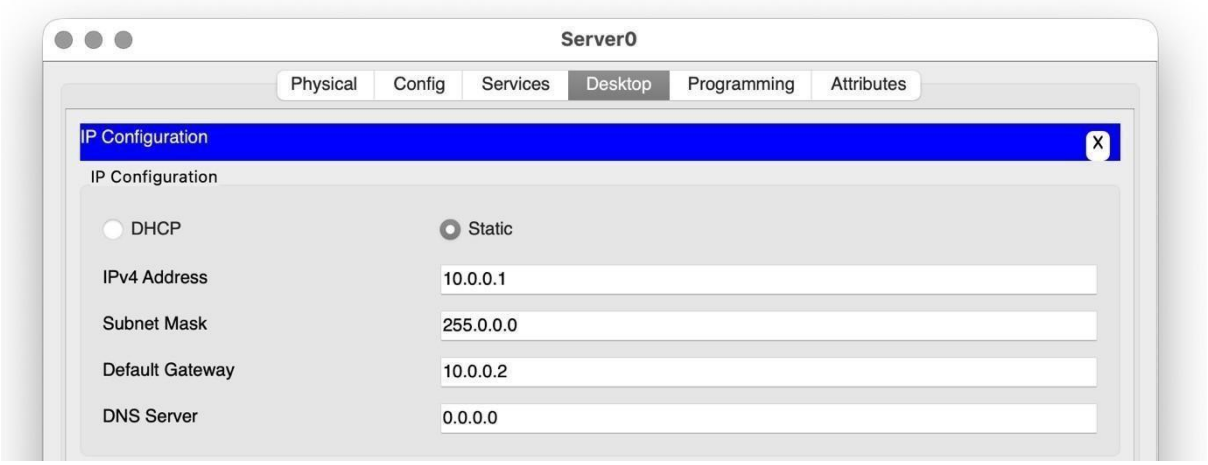


fig (4.5) IP addressing of PC

- Go to Router's CLI and assign IP addresses to each Interface.

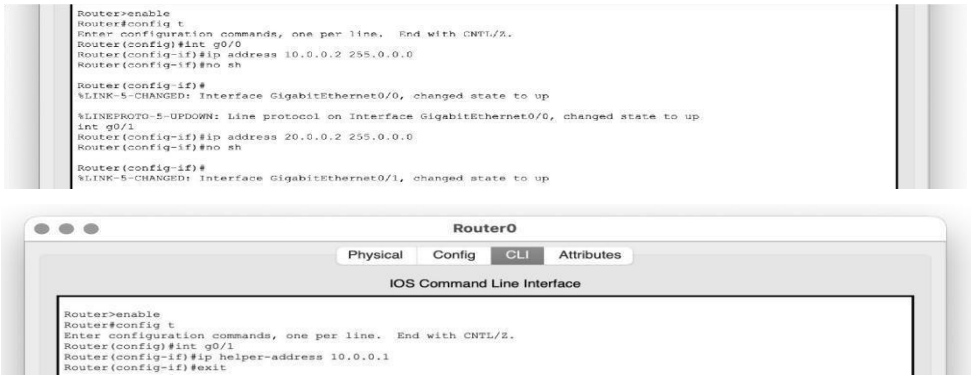


fig (4.6) CLI Commands

- Go to Services -> DHCP and set up 2 DHCP Pools and turn ON the service.

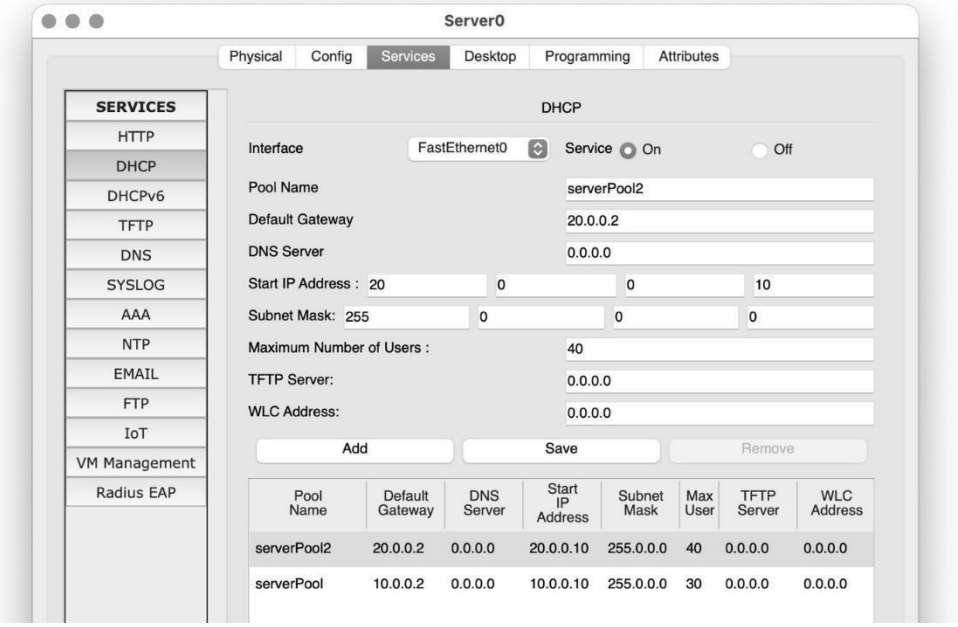


fig (4.7) DHCP Services

Step 3: Configure the End Devices to Use DHCP.

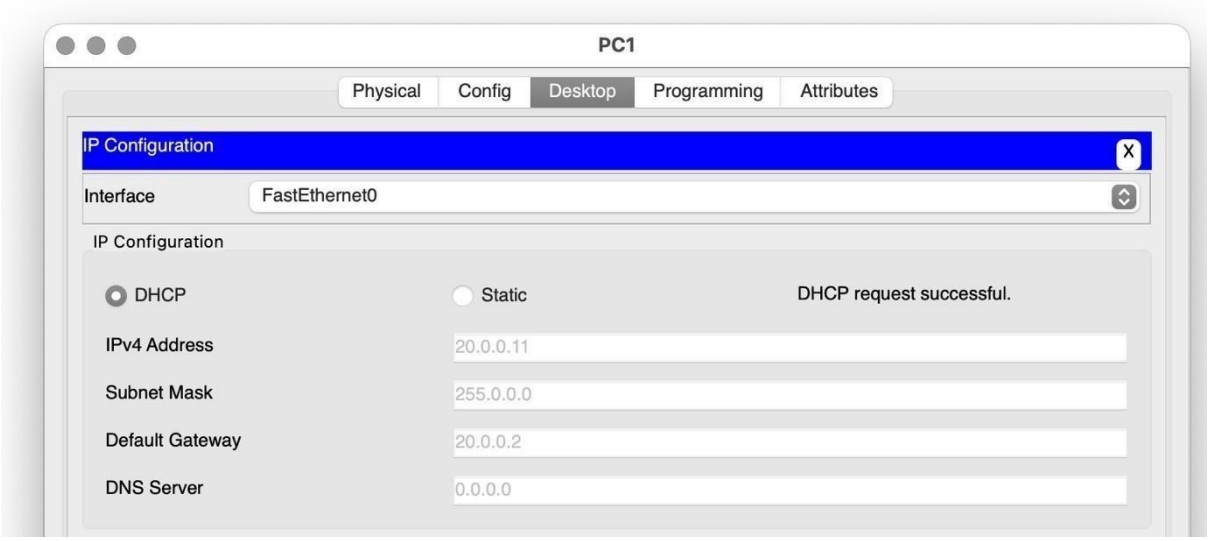


fig. (4.8) Switching to DHCP from Static IP addressing

(C). Objective: To configure a DHCP server on a router. Assign IP addresses dynamically to devices on the network and verify successful address assignment. (Using packet Tracer)

Step 1: Device Setup and Connections

1. Open Packet Tracer.
2. Add the required devices (routers, switch and end devices) to the workspace.
3. Use straight-through Ethernet cables to connect the devices.

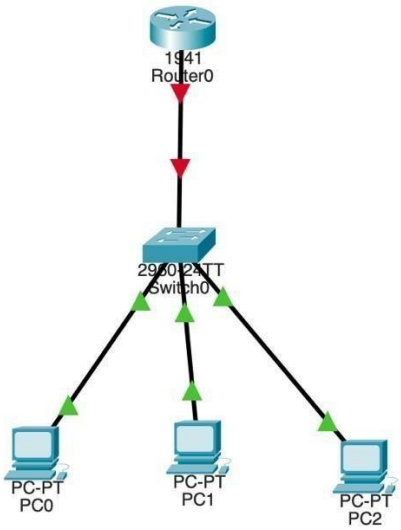


fig (4.9) Configuration of DCHP Server **Step 2: Configure the Router as a DHCP Server** 1. Assign an IP address to the router and access its CLI.

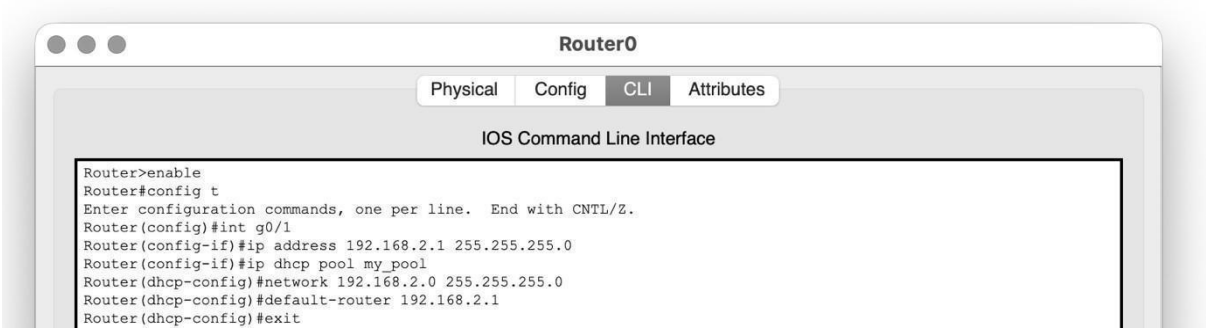
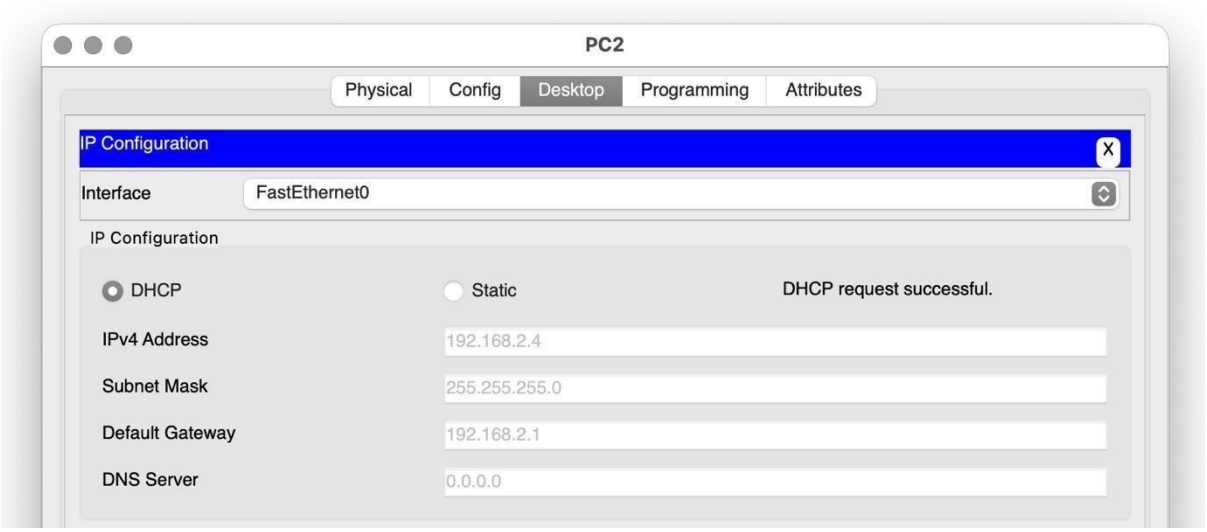


fig (4.10) CLI Commands on Router

Step 3: Verify DHCP Configuration

1. On each end device, go to Desktop -> IP Configuration.
2. Select the DHCP option to request an IP address from the DHCP server.
3. Verify that each device receives an IP address from the DHCP pool and check for successful assignment.



fig

(4.11) Switching to DHCP from Static IP addressing

\

Experiment No. 5

Objective: To configure a local DNS server to resolve domain names within a network. (Using packet Tracer).

Procedure:

Step 1: Set Up the Devices

1. Open Packet Tracer.
2. Add the following devices to the workspace:
    - 2 End Devices (e.g., PCs)
    - 1 Switch • 2 Server

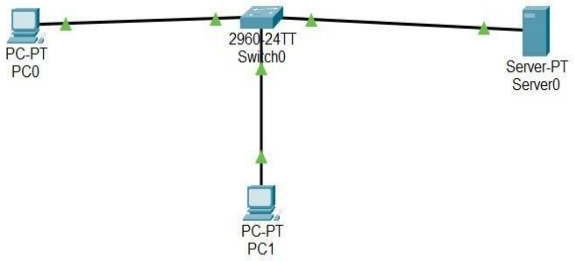


fig. (5.1) network topology **Step 2: Assign a static IP address to the server**

- Click on the server1, open the "IP Configuration."
- Set the IPv4 Address to 192.168.1.2.
- Ensure the Subnet Mask is 255.255.255.0.

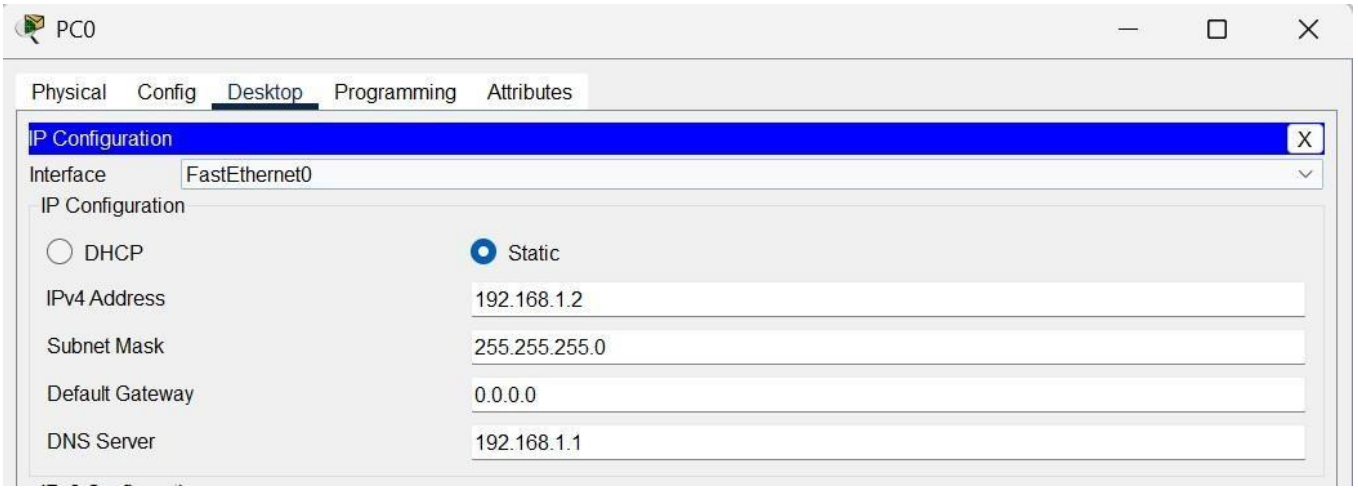


Fig. (5.2) Static IP to server

Step 3: Assign DNS server IP on the end devices :

- On each desktop, open the "Desktop" tab, select "IP Configuration," and set the DNS server IP to 192.168.1.1.

- Desktop > ip config > DNS server (192.168.2.1)

Step 4: Configure DNS Settings on the Server

1) Click on the server, go to the "Services" tab, and select "DNS." 2) Add DNS records by specifying the following:

- Name: cnwebsite.com
- Address: 192.168.1.2

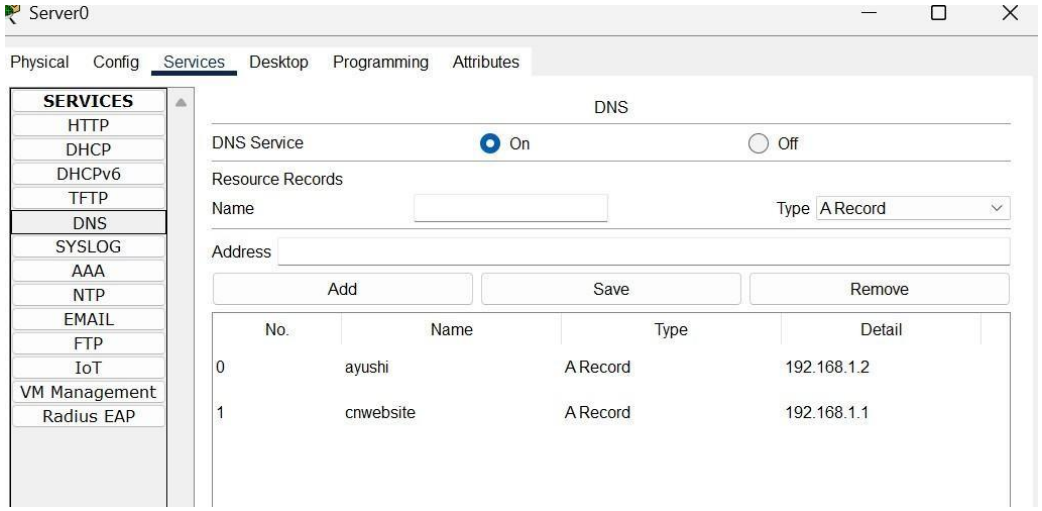


Fig. (5.3) DNS services

Step 5: Configure the Web Server:

1. Select the web server.
2. Go to the Desktop tab and click on IP Configuration. Assign:
   - **IP Address**: 192.168.2.2
   - **Subnet Mask**: 255.255.255.0
3. Go to the Services tab, and click on HTTP. Ensure the HTTP service is ON.

Step 6: Test the DNS Server Configuration : 1)

Verify DNS resolution:

- On each desktop, open the web browser.
- Enter the domain name (dns.com) in the address bar.
- Ensure that the web page loads correctly, indicating that DNS resolution is working.

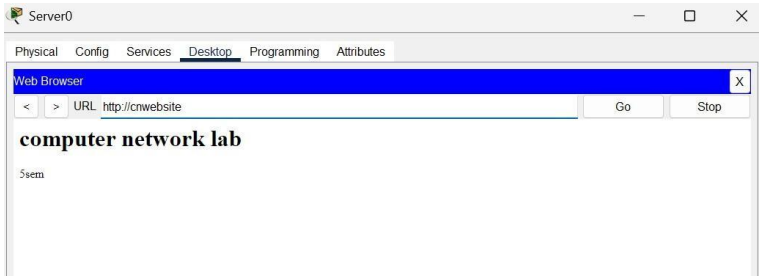


Fig. (5.4) Testing via domain name

Experiment: 6

Objective: Static Routing: Configure static routes on multiple routers to enable communication between different networks. Test the connectivity by pinging between hosts in different networks. (Using packet Tracer).

Introduction to Static Routing

Static routing is a network routing method where routes are manually configured by a network administrator, rather than dynamically calculated by a routing protocol. This means that once a static route is set, it remains fixed unless manually updated by the administrator, unlike dynamic routing which can automatically adapt to network changes. **Network Topology Overview:**

- Two routers: **Router0** and **Router1**
- Two switches: **Switch0** and **Switch1**
- Four PCs: **PC0, PC1, PC2, PC3**
- PC0 and PC1 are in the network connected to **Router0**
- PC2 and PC3 are in the network connected to **Router1** • Routers are connected via copper cross-wire.

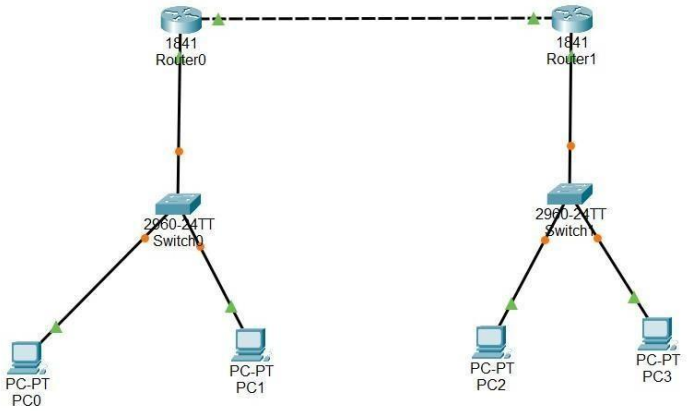


Fig. (6.1) Network Topology

Step 1: Configuration PC	**IP Address**	**Subnet Mask**	**Default Gateway**
PC0:	192.168.2.7	255.255.255.0	192.168.2.3
PC1:	192.168.2.9	255.255.255.0	192.168.2.3
PC2:	192.168.3.5	255.255.255.0	192.168.3.3
PC3:	192.168.3.7	255.255.255.0	192.168.3.3

Router	**Interface**	**IP Address**	**Subnet Mask**
Router0	Gig0/0/0	192.168.2.3	255.255.255.0
Router0	Gig0/0/1	192.168.1.2	255.255.255.0
Router1	Gig0/0/0	192.168.3.3	255.255.255.0
Router1	Gig0/0/1	192.168.1.4	255.255.255.0

Step 2: Set Static Routes

Router0 > Config tab > Static Routing:

- Network: 192.168.3.0
- Subnet Mask: 255.255.255.0
- Next Hop: 192.168.1.4
- Click Add

Router1 > Config tab > Static Routing:

- Network: 192.168.2.0
- Subnet Mask: 255.255.255.0
- Next Hop: 192.168.1.2
- Click Add

Step 3: Test Connectivity

- Use the ping tool or Command Prompt in PC0 and ping PC2 or PC3.
- On PC0, go to Desktop > Command Prompt: ping 192.168.3.5

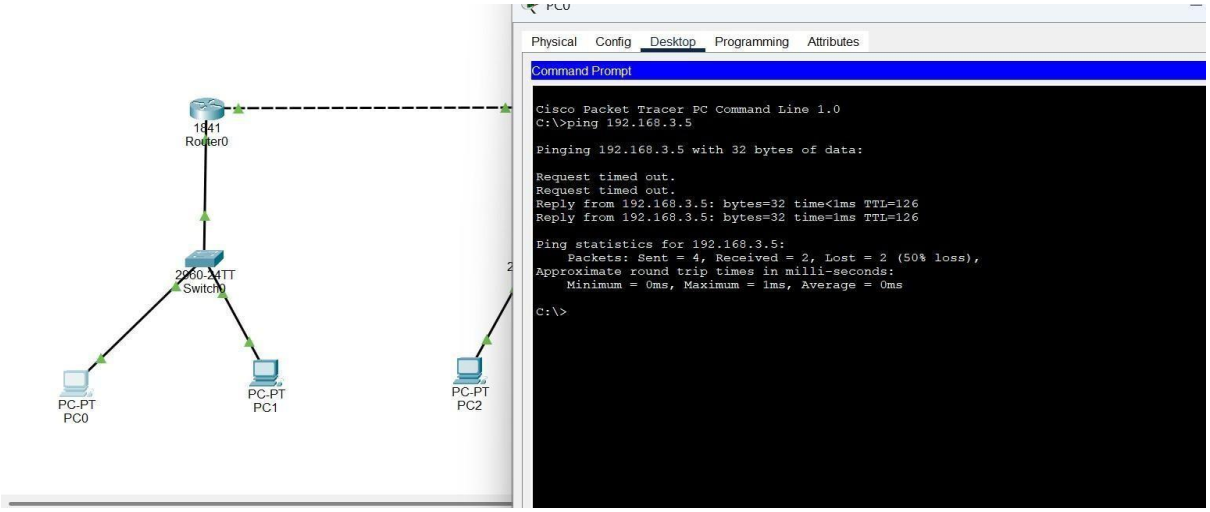


Fig. (6.2) Test Connectivity

Result:

- If everything is configured correctly, you should see successful replies.
- Static routing has been successfully implemented using the GUI without CLI.

Conclusion

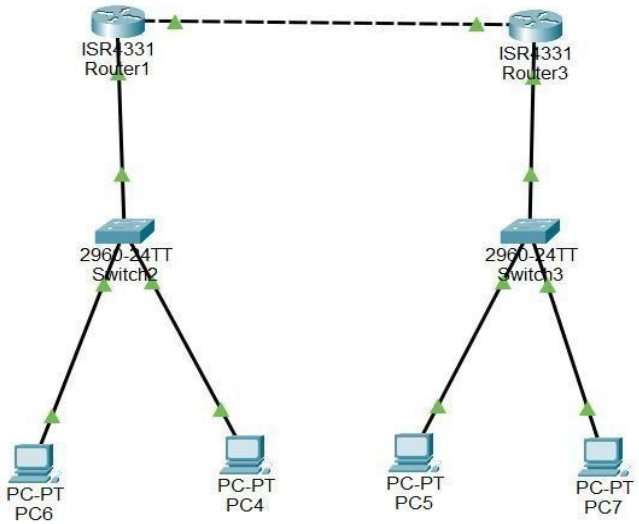
Static routing can be easily implemented without CLI by configuring IPs and routes using the GUI interface in Cisco Packet Tracer. It enables communication between devices in different networks through manual route entries.

Experiment:7

Objective: Dynamic Routing (RIP): Configure routers to use the Routing Information Protocol (RIP) for dynamic routing. Enable RIP on the interfaces connected to different networks and verify that routes are being learned and propagated.

Step 1: Set Up the Devices

1. Open Packet Tracer.
2. Add the following devices to the workspace:
   - 2 Routers
   - 2 Switches
   - 4 End Devices



fig

(7.1) Configuration for RIP

Step 2: Assign IP Addresses to Devices

1. Configure IP addresses for each router interface:
   Router 1:
   - Interface 1 (connected to Network 1): 192.168.1.0 • Interface 2 (connected to Router1): 11.0.0.1 Router 2:
   - Interface 1 (connected to Router0): 11.0.0.2
   - Interface 2 (connected to Network 2): 192.168.2.0

2. Assign IP addresses to PCs:
   Network 1:
   - PC0: 192.168.1.1
   - PC1: 192.168.1.2

   Network 2:
   - PC3: 192.168.2.1
   - PC4: 192.168.2.2

Step 3: Configure RIP on Routers

1. On Router0:

Open the config -> RIP Add all networks in it.

- 10.0.0.0 • 20.0.0.0

2. On Router1:

Open the config -> RIP Add all networks in it.

- 30.0.0.0
- 20.0.0.0

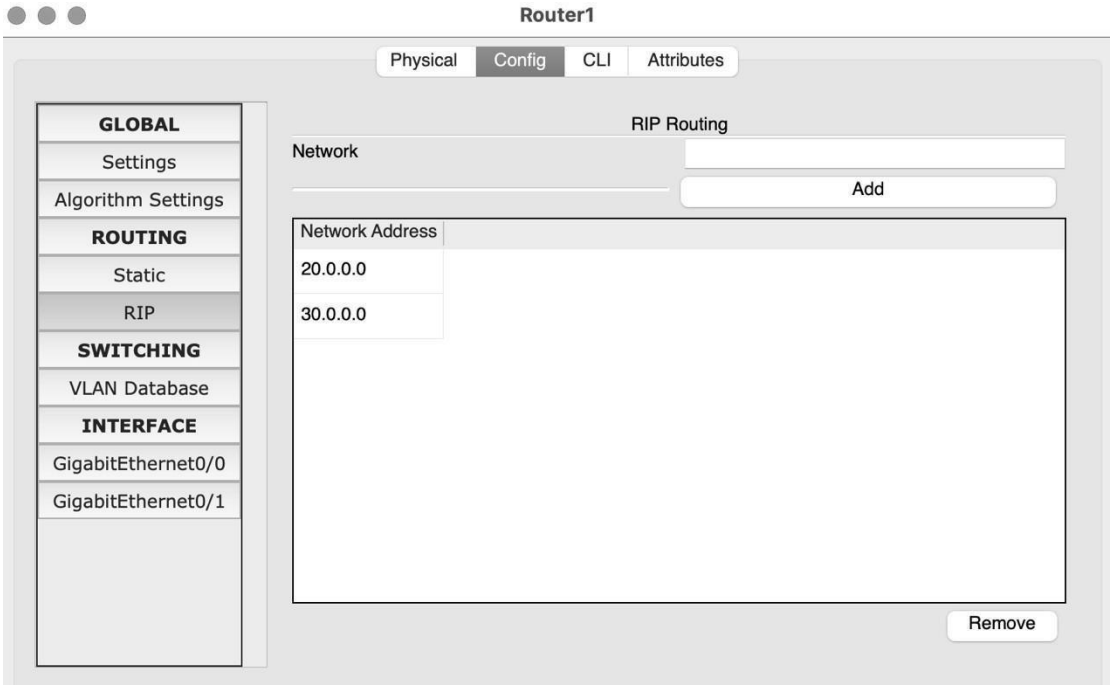


fig (7.2) RIP Configuration

Step 4: Verifying by sending packet from one network to another.

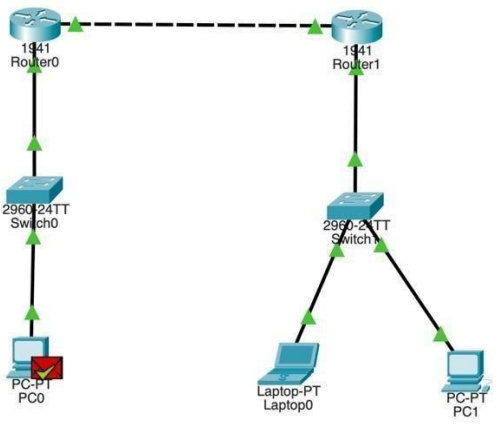


fig (7.3)

Verifying Network

Experiment No. 8

Objective: Static Routing (OSPF): Configure routers to use the Open Shortest Path First (OSPF) routing protocol. Set up OSPF on the routers and advertise network information. Verify that OSPF is establishing neighbour relationships and propagating routes. Test connectivity between hosts in different networks. (Using packet Tracer) **Step 1: Set Up the Devices**

1. Open Packet Tracer.
2. Add the following devices to the workspace:
    - 3 Routers
    - 3 Switches
    - 6 End Devices (2 PCs for each network)

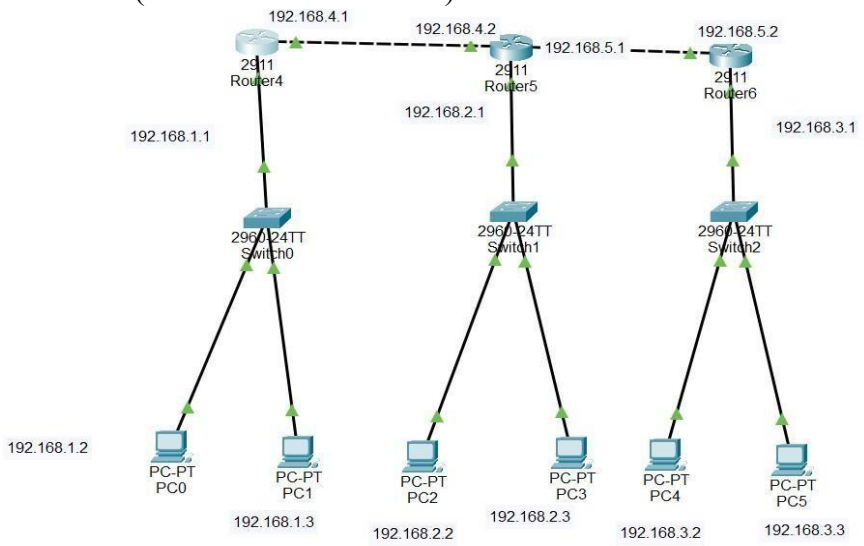


fig (8.1) Configuration for OSPF

Step 2: Assign IP Addresses to Devices

1. Configure IP addresses for each router interface:

Router0:

- Interface 1 (connected to Network 1): 10.0.0.1
- Interface 2 (connected to Router1): 40.0.0.1 Router1:
- Interface 1 (connected to Network 2): 20.0.0.1
- Interface 2 (connected to Router0): 40.0.0.2
- Interface 3 (connected to Router2): 50.0.0.1 Router 2:
- Interface 1 (connected to Network 3): 30.0.0.1
- Interface 2 (connected to Router1): 50.0.0.2

2. Assign IP addresses to PCs: Network 1:
    - PC0: 10.0.0.2
    - PC1: 10.0.0.3 Network 2:
    - PC2: 20.0.0.2
    - PC3: 20.0.0.3 Network 3:

- PC4: 30.0.0.2
- PC5: 30.0.0.3

Step 3: Configure OSPF on Routers

1. On Router0: Open the CLI tab.

```
Router>
Router>
Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router ospf 1
Router(config-router)#network 192.168.1.0 0.0.0.255 area 1
Router(config-router)#network 192.168.4.0 0.0.0.255 area 1
```

fig (8.2) CLI for Area 1 in Router 0

2. On Router2:
   Open the CLI tab.

```
Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z
Router(config)#router ospf 1
Router(config-router)#network 192.168.3.0 0.0.0.255 area 2
Router(config-router)#network 192.168.5.0 0.0.0.255 area 2
```

fig (8.3) CLI for Area 2 in Router 1

3. On Router1:
   Open the CLI tab.

```
Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#router ospf 1
Router(config-router)#network 193.168.2.0 0.0.0.255 area 0
Router(config-router)#network 193.168.4.0 0.0.0.255 area 0
Router(config-router)#network 193.168.5.0 0.0.0.255 area 0
```

fig (8.4) CLI for Area 0 in Router 1

Step 5: Test Connectivity

1. On each PC, open the Command Prompt.
2. Use the ping command to test connectivity between hosts.
3. Verify successful communication with replies to the ping request.

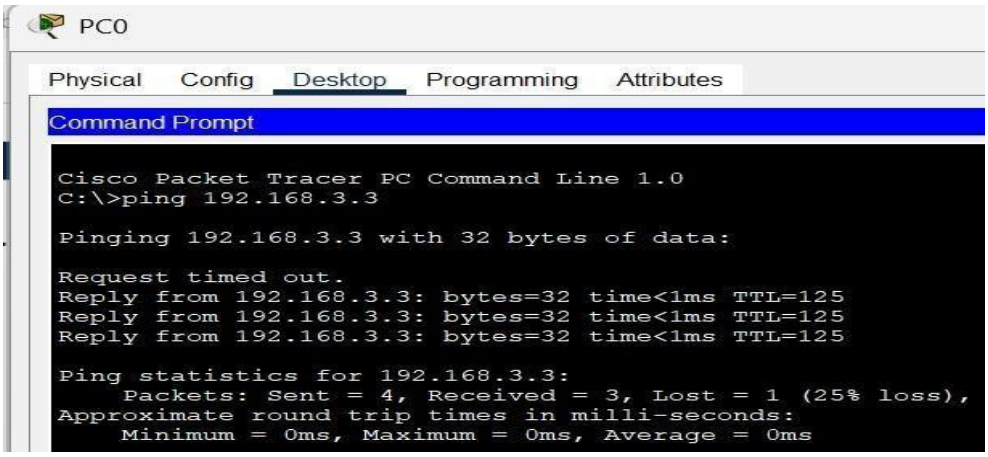


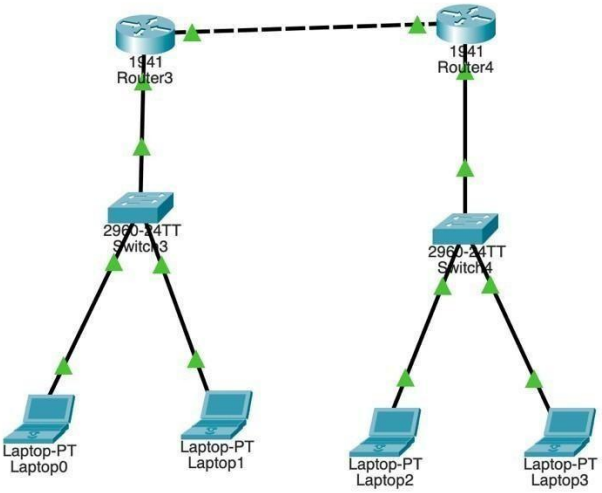
fig (8.5) Checking Connectivity

Experiment 9

Objective: NAT (Network Address Translation): Set up NAT on a router to translate private IP addresses to public IP addresses for outbound internet connectivity. Test the translation and examine how NAT helps conserve IPv4 address space. (Using packet Tracer)

Step 1: Set Up the Devices

1. Open Packet Tracer.
2. Add the following devices to the workspace:
    - 2 Routers
    - 2 Switches
    - 4 End Devices (2 Laptops for each network)



fig

(9.1) Configuration for NAT

Step 2: Assign IP Addresses to Devices

1. Configure IP addresses for each router interface:

Router3:

- Interface 1 (connected to Network 1): 10.0.0.1
- Interface 2 (connected to Router4): 30.0.0.1 Router4:
- Interface 1 (connected to Network 2): 20.0.0.1
- Interface 2 (connected to Router3): 30.0.0.2

2. Assign IP addresses to PCs: Network 1:
    - Laptop0: 10.0.0.2
    - Laptop1: 10.0.0.3 Network 2:
    - Laptop2: 20.0.0.2
    - Laptop3: 20.0.0.3

Step 3: Configure routers to use the RIP for dynamic routing.

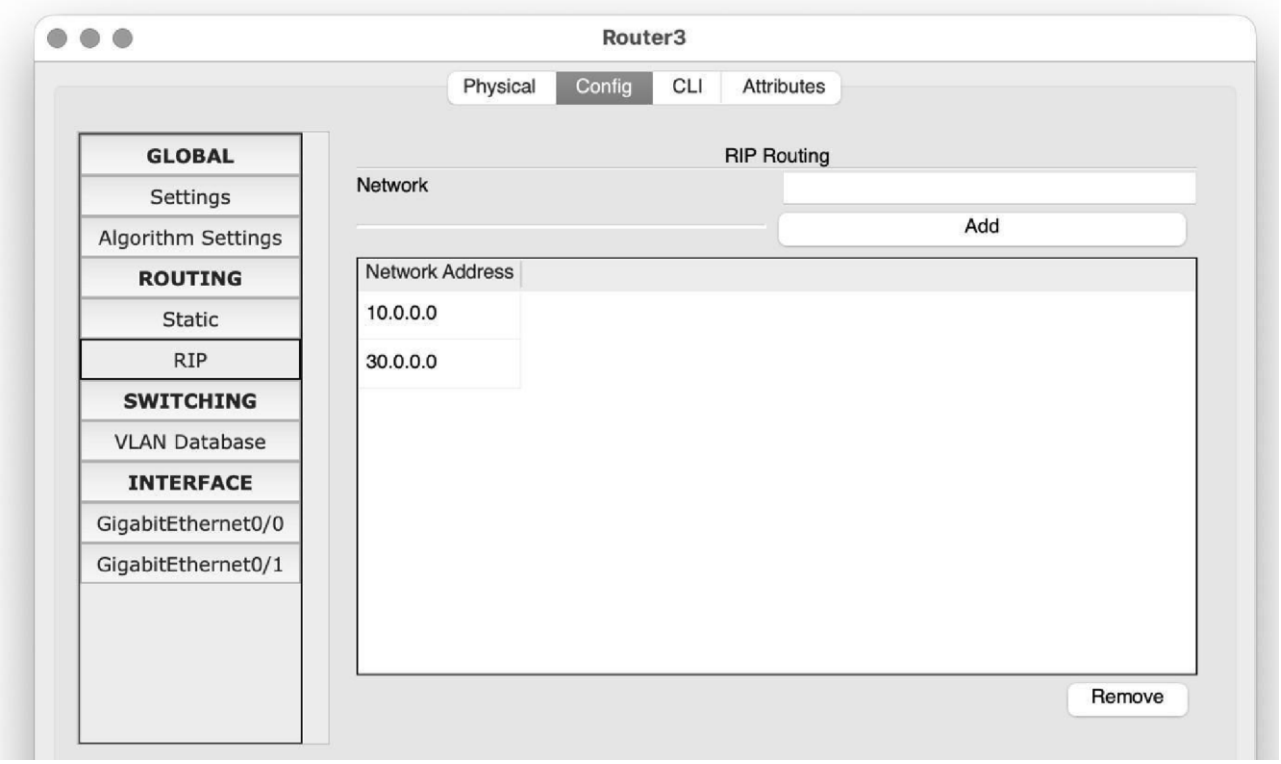


fig (9.2) Adding networks in Router **Step 4: Go to Router3:** Open the CLI tab

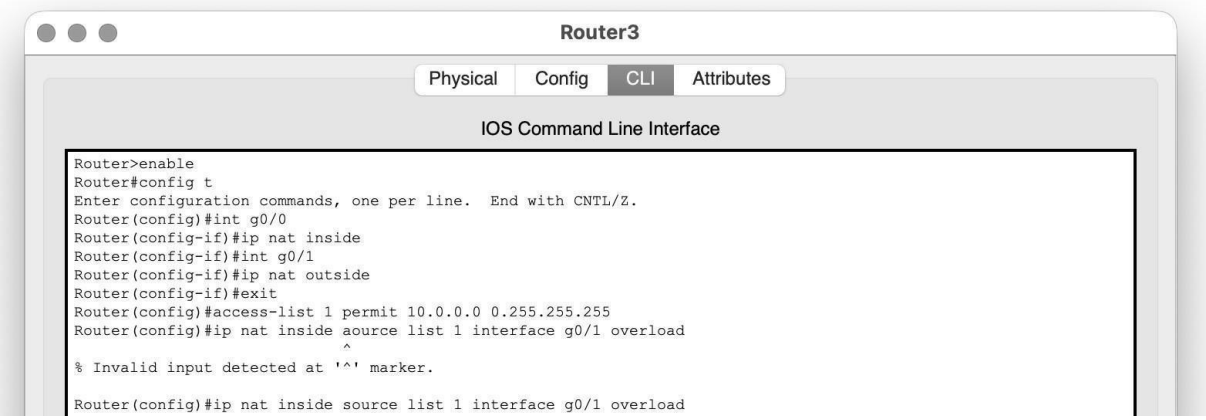


fig (9.3) CLI Commands

Step 5: Send packet from one end device of Network 1 to end device of Network 2

Open the Router3 CLI:

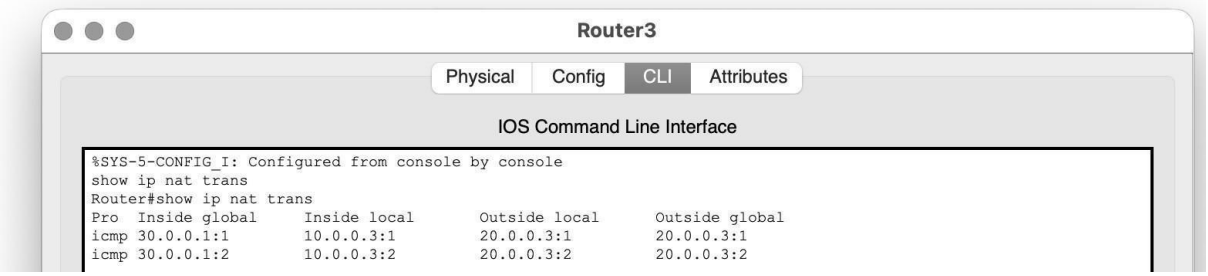


fig (9.4) Displaying Transmission

Experiment 10

Objective: To monitor network traffic using WireShark and to analyze complete TCP/IP protocol suite layer's headers using WireShark**.**

Step 1: Open WireShark and Click on WI-Fi: en0

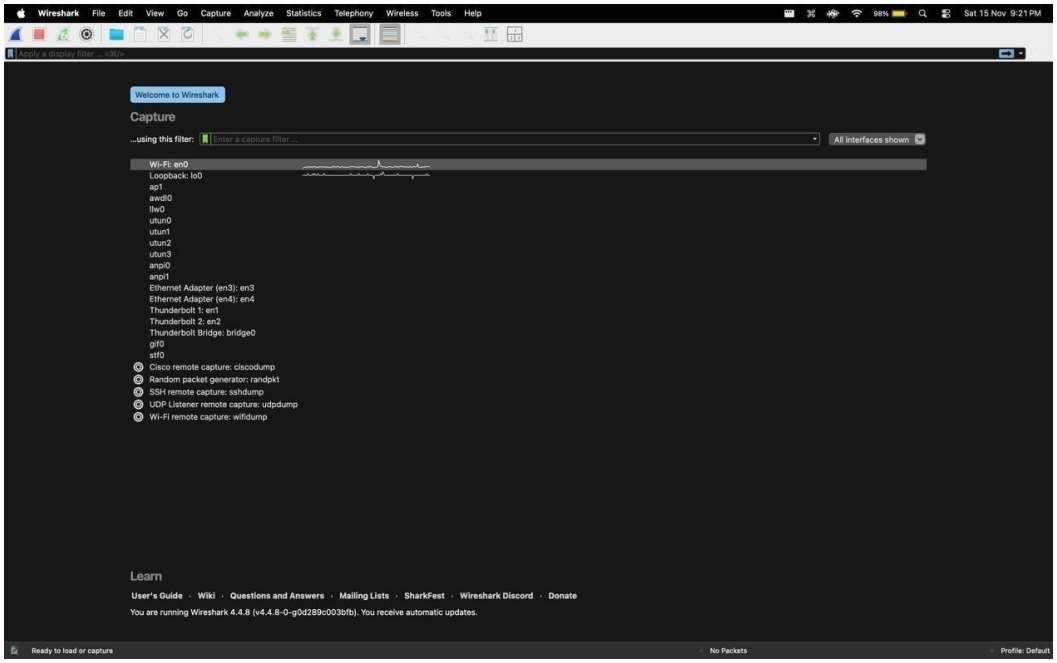


fig (10.1) Selecting Ethernet Connection

Step 2: Go to Capture->Stop and filter by "tcp"

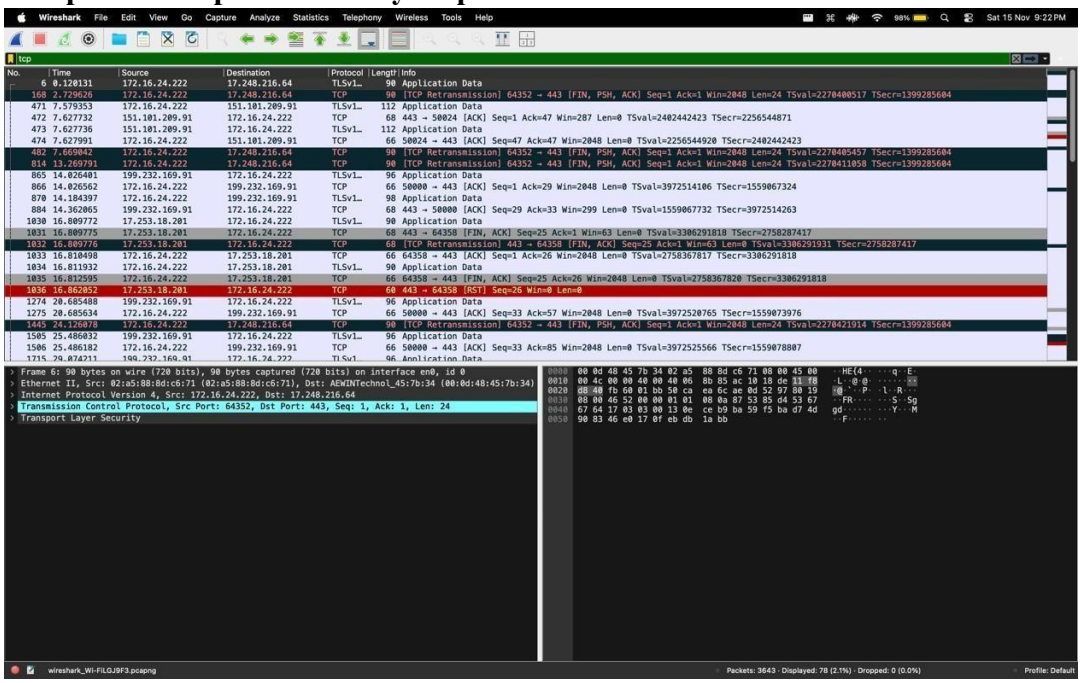


fig (10.2) Displaying network Traffic

Experiment:11

Objective:
Write a TCP server in C that listens on a port, accepts a client connection, and prints the received message.
Write a TCP client that connects to the server and sends a message. Test the communication on the same machine or over a network.

TCP Server in C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080  // Define the port the server will listen on
int main() {       int server_fd, new_socket;       struct
sockaddr_in address;       int addrlen = sizeof(address);       char
buffer[1024] = {0};

        if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
                        perror("Socket failed");
                exit(EXIT_FAILURE);
        }

        // Set server address information       address.sin_family = AF_INET;  // IPv4
        address.sin_addr.s_addr = INADDR_ANY;  // Bind to all available interfaces
        address.sin_port = htons(PORT);  // Convert to network byte order

        // Bind the socket to the specified port       if (bind(server_fd, (struct
        sockaddr *)&address, sizeof(address)) < 0) {
                perror("Bind failed");          close(server_fd);          exit(EXIT_FAILURE);
        }

        // Start listening for connections (up to 3 in queue)       if
        (listen(server_fd, 3) < 0) {          perror("Listen failed");
        close(server_fd);
                exit(EXIT_FAILURE);
        }

          printf("Server is listening on port %d...\n",
        PORT);
        // Accept a new connection       if ((new_socket = accept(server_fd, (struct sockaddr
```

```
*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    // Read message from the client    int
    valread = read(new_socket, buffer, 1024);
    printf("Received message: %s\n", buffer);

    // Close the socket after communication    close(new_socket);
    close(server_fd);

    return 0;
}
```

TCP Client in C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080  // The port number the client will connect to

int main() {    int sock = 0;    struct sockaddr_in serv_addr;
    char *message = "Hello from the client!";  // Message to
    be sent    char buffer[1024] = {0};  // Buffer to store
    server response

    // Create a socket (IPv4, TCP, IP)    if ((sock =
    socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    printf("\nSocket creation error\n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;    serv_addr.sin_port = htons(PORT);
    // Convert port to network byte order



    // Convert server IP address to binary form (localhost here)    if
    (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
```

```
        printf("\nInvalid address/ Address not supported\n");          return -
        1;
        }

        // Connect to the server       if (connect(sock, (struct sockaddr
        *)&serv_addr, sizeof(serv_addr)) < 0) {        printf("\nConnection
                Failed\n");
                return -1;
        }

        // Send message to the server       send(sock,
        message, strlen(message), 0);
        printf("Message sent from client: %s\n", message);

        // Optionally, receive a message back from the server (if needed)
        // int valread = read(sock, buffer, 1024);
        // printf("Server response: %s\n", buffer);

        // Close the socket after communication       close(sock);
        return 0;
}
```

Testing the Programs

Compile the Server and Client

$ gcc -o server server.c

$ gcc -o client client.c

Fig. (11.1) TCP server

The client will output: Message sent from client: Hello from the client!

Experiment:12

Objective:
UDP Client-Server Communication:
Implement a UDP client program that sends a message to a UDP server program.
Implement the corresponding UDP server program that receives the message and displays it. (Using 'C' Language)

UDP Server in C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080  // Define the port the server will listen on

int main() {      int sockfd;      char buffer[1024];      struct
        sockaddr_in server_addr, client_addr;      socklen_t
        len;      int n;
        // Create a UDP socket      if ((sockfd = socket(AF_INET,
        SOCK_DGRAM, 0)) < 0) {
                perror("Socket creation failed");
                exit(EXIT_FAILURE);
        }

        // Set server address information      memset(&server_addr, 0,
        sizeof(server_addr));      memset(&client_addr, 0,
        sizeof(client_addr));

        server_addr.sin_family = AF_INET;  // IPv4      server_addr.sin_addr.s_addr =
        INADDR_ANY;  // Bind to all available interfaces      server_addr.sin_port =
        htons(PORT);  // Convert to network byte order

        // Bind the socket to the port      if (bind(sockfd, (const struct sockaddr
        *)&server_addr, sizeof(server_addr)) < 0) {
                perror("Bind failed");          close(sockfd);          exit(EXIT_FAILURE);
        }

        printf("UDP Server is listening on port %d...\n", PORT);


        len = sizeof(client_addr);  // Size of the client address
```

```
        // Wait for a message from the client       n = recvfrom(sockfd, (char *)buffer, 1024,
        MSG_WAITALL, (struct sockaddr *)&client_addr,
&len);      buffer[n] = '\0';  // Null-terminate the received
        message

        printf("Received message from client: %s\n", buffer);

        // Close the socket       close(sockfd);
        return
        0;
}
```

UDP Client in C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080  // The port number the client will send data to
#define SERVER_IP "127.0.0.1"  // The server IP address (localhost here)

int main() {       int sockfd;      char *message
        = "Hello
        from the client!";      struct sockaddr_in
        server_addr;

        // Create a UDP socket       if ((sockfd = socket(AF_INET,
        SOCK_DGRAM, 0)) < 0) {
                perror("Socket creation failed");
                exit(EXIT_FAILURE);
        }

        // Set server address information       memset(&server_addr, 0,
        sizeof(server_addr));       server_addr.sin_family = AF_INET;
        server_addr.sin_port = htons(PORT);  // Convert port to network byte order
        server_addr.sin_addr.s_addr = inet_addr(SERVER_IP);  // Convert server IP to binary form

        // Send the message to the server       sendto(sockfd, (const char *)message, strlen(message),
        MSG_CONFIRM, (const struct sockaddr
*)&server_addr,  sizeof(server_addr));      printf("Message
        sent from client: %s\n", message);
```

```
    // Close the socket
    close(sockfd);
    return 0;
}
```

Testing the Programs

Compile the Server and Client:
$ gcc -o udp_server udp_server.c
$ gcc -o udp_client udp_client.c

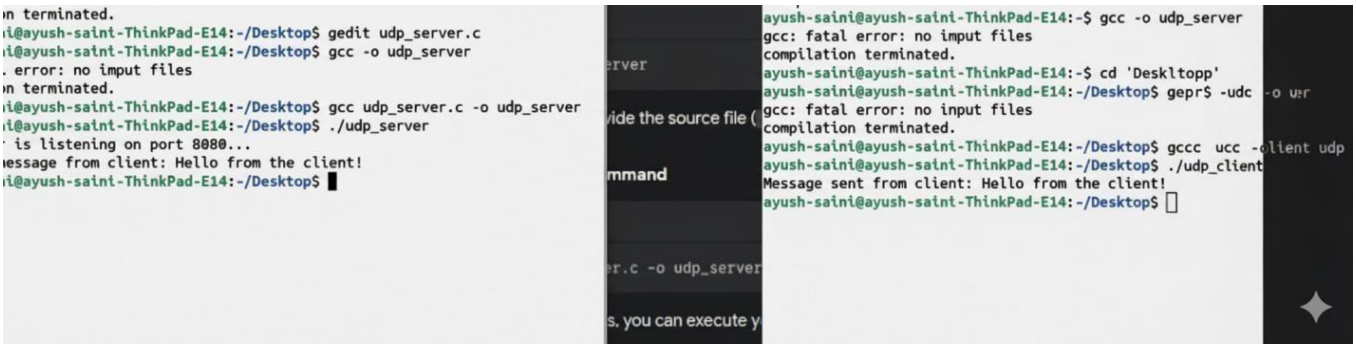


Fig. (12.1) UDP Server

The client will output: Message sent from client: Hello from the client!